Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

AutoAudit

Sistema de auditoría automatizado

Documentación técnica del proyecto · Bases de Datos I

Catedrático	René Ornélyz
Integrantes	Carlos Mendez — 9490-24-17379 Maycol Ortiz — 9490-24-27213
Repositorio	https://github.com/Gaston1x/autoaudit
Plataforma	PostgreSQL 18 · Ubuntu 26.04.1 LTS · Python
Fecha	noviembre de 2026

Contenido

1. Descripción general y objetivos
2. Arquitectura de la solución
3. Modelo entidad-relación
   - Diagrama
   - Entidades y relaciones
4. Diccionario de datos
5. Funcionamiento interno
   - Consultas al diccionario del DBMS
   - Generación dinámica de triggers
   - Bitácora
   - Gestión de dependencias
   - Consultas parametrizadas
6. Manual de configuración y activación de auditoría por tabla
7. Procedimiento de instalación y despliegue de triggers
8. Seguridad y privacidad
9. Manejo de errores
10. Pruebas de validación
    - Pruebas funcionales
    - Pruebas de rendimiento
11. Matriz de cumplimiento
12. Anexo: estructura del repositorio y API

1. Descripción general y objetivos

AutoAudit es un sistema de auditoría configurable para PostgreSQL. Un administrador elige, a partir del diccionario de datos, qué tablas, qué columnas y qué operaciones (INSERT, UPDATE, DELETE) deben vigilarse. Con esa configuración el sistema genera automáticamente, en PL/pgSQL, los triggers que registran cada modificación en una bitácora centralizada, con el usuario responsable, la estación de trabajo, la fecha y hora y los valores anterior y nuevo de cada columna.

Objetivo principal

Desarrollar una solución de auditoría automática que capture cambios en tablas configurables mediante la generación dinámica de triggers, permitiendo la consulta histórica de modificaciones por usuario o por columna específica.

Objetivos específicos

Objetivo	Cómo se cumple
Diseñar el modelo entidad-relación completo del sistema de auditoría	Esquema `autoaudit` con 12 tablas (sección 3)
Configuración de tablas y columnas basada en el diccionario de datos	Funciones sobre `pg_catalog` y pantalla de configuración (secciones 5.1 y 6)
Generación automática de triggers para INSERT, UPDATE y DELETE	Generador PL/pgSQL `_fn_generar_trigger` (sección 5.2)
Bitácora con información completa de cada cambio	Tablas `aud_bitacora` y `aud_bitacora_detalle` (sección 5.3)
Consulta histórica por usuario o por columna	Funciones parametrizadas, web y CLI con exportación (sección 5.5)

2. Arquitectura de la solución

La lógica de auditoría vive dentro de la base de datos: así ningún cambio escapa, sin importar desde qué aplicación se haga. Python se usa para la interfaz web, la línea de comandos, los reportes y las pruebas.

PostgreSQL 18 (PL/pgSQL)

- Esquema `autoaudit`: configuración, catálogos y bitácora.
- Consultas al diccionario de datos (`pg_catalog`).
- Generador dinámico de triggers.
- Event triggers para dependencias DDL.
- Funciones de consulta parametrizadas.
- Roles y permisos (seguridad).

Python 3

- Web: FastAPI + Jinja2 + Bootstrap.
- Acceso a datos: psycopg 3 (consultas parametrizadas).
- CLI: Typer (`cli.py`).
- Reportes: CSV, Excel (openpyxl), PDF (reportlab), JSON.
- Pruebas: pytest; rendimiento con matplotlib.
- Documentación: este PDF se genera desde el sistema.

Versión de la base de datos usada: PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, 64-bit.

Flujo de trabajo

1. El administrador consulta las tablas existentes (diccionario de datos) y marca columnas y eventos.
2. La configuración se valida contra el catálogo y se guarda en `aud_tabla`, `aud_columna` y `aud_configuracion`.
3. El generador construye una función PL/pgSQL y un trigger `AFTER … FOR EACH ROW` específicos para esa tabla y los ejecuta.
4. Cada INSERT, UPDATE o DELETE de cualquier usuario dispara el trigger, que escribe la cabecera y el detalle en la bitácora.
5. Auditores y administradores consultan el historial por usuario, por columna o por registro, y lo exportan.

3. Modelo entidad-relación

3.1 Diagrama

Diagrama generado automáticamente a partir de las llaves foráneas reales del esquema `autoaudit`. Las flechas van de la tabla referenciada a la tabla que la referencia (uno a muchos).

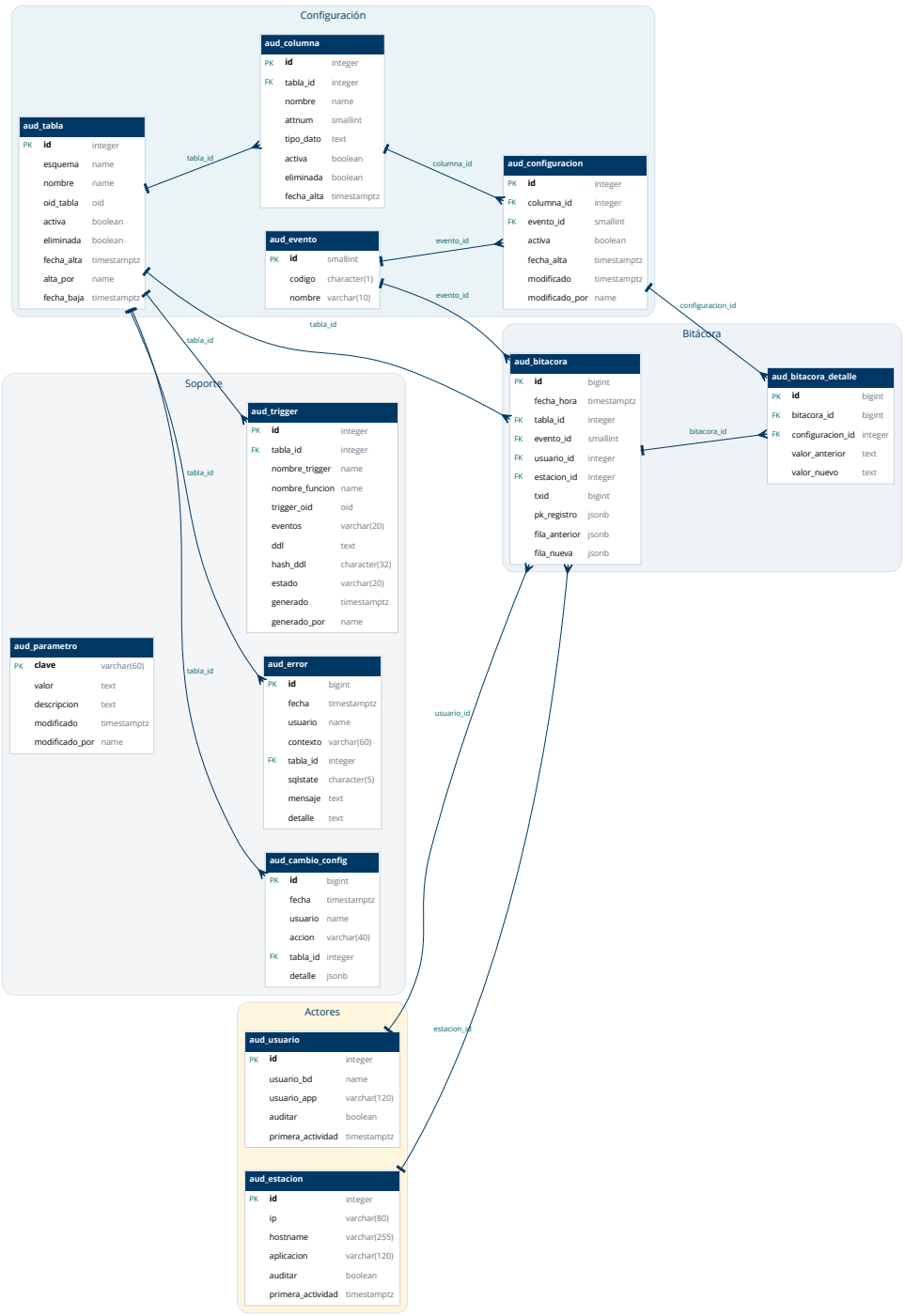


Figura 1. Modelo entidad-relación del esquema autoaudit (versión ampliable en `docs/img/er_autoaudit.svg`).

3.2 Entidades y relaciones

Entidad requerida	Tabla	Descripción
Tablas auditadas	`aud_tabla`	Tablas de negocio registradas; se identifican por OID para sobrevivir a renombres.
Columnas auditadas	`aud_columna`	Columnas de cada tabla, identificadas por `attnum`.
Eventos por columna y tabla	`aud_evento`, `aud_configuracion`	Catálogo de operaciones y la combinación columna × evento que debe auditarse.
Usuarios auditados	`aud_usuario`	Usuario de base de datos y, opcionalmente, usuario de la aplicación. Se puede excluir.
Estaciones de trabajo auditadas	`aud_estacion`	IP (según la política de privacidad), nombre de la estación y aplicación.

Relaciones exigidas

- **La configuración referencia a los eventos que se deben registrar ante el cambio de una columna de una tabla:** `aud_configuracion.evento_id → aud_evento` y `aud_configuracion.columna_id → aud_columna.tabla_id → aud_tabla`.
- **Cada registro de bitácora referencia a la configuración usada, al usuario y a la estación:** `aud_bitacora_detalle.configuracion_id → aud_configuracion`, `aud_bitacora.usuario_id → aud_usuario` y `aud_bitacora.estacion_id → aud_estacion`.

> **Normalización.** La bitácora tiene cabecera (una fila por operación) y detalle (una fila por columna afectada), en tercera forma normal. La consulta por columna usa el índice `(configuracion_id, bitacora_id)` del detalle y la consulta por usuario el índice `(usuario_id, fecha_hora)` de la cabecera.

> **Integridad referencial y preservación de datos.** La bitácora no tiene llave foránea hacia las filas de negocio (que pueden borrarse), sino hacia la configuración, que nunca se borra físicamente: se desactiva. Así la bitácora mantiene integridad referencial con las tablas configuradas y conserva los datos aunque el registro original se elimine.

4. Diccionario de datos

Generado desde el catálogo de PostgreSQL con `autoaudit.fn_diccionario_datos('autoaudit')`; las descripciones provienen de los `COMMENT ON` de cada objeto.

aud_tabla

Tablas de negocio registradas para auditoría

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador interno
2	**esquema**	name	NO			Esquema de la tabla origen
3	**nombre**	name	NO			Nombre de la tabla origen
4	**oid_tabla**	oid	NO	UQ		OID de la tabla en pg_class (sobrevive a renombres)
5	**activa**	boolean	NO		true	true si la auditoría de la tabla está habilitada
6	**eliminada**	boolean	NO		false	true si la tabla origen fue eliminada (DROP TABLE)
7	**fecha_alta**	timestamp with time zone	NO		now()	Fecha en que se registró la tabla
8	**alta_por**	name	NO		SESSION_USER	Administrador que registró la tabla
9	**fecha_baja**	timestamp with time zone	SÍ			Fecha en que se desactivó o eliminó

aud_columna

Columnas de las tablas auditadas

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador interno
2	**tabla_id**	integer	NO	UQ, FK	autoaudit.aud_tabla	Tabla a la que pertenece (aud_tabla)
3	**nombre**	name	NO			Nombre actual de la columna
4	**attnum**	smallint	NO	UQ		Número de columna en pg_attribute (sobrevive a renombres)
5	**tipo_dato**	text	NO			Tipo de dato según el diccionario de datos

6	**activa**	boolean	NO	true	true si al menos un evento puede auditarse en la columna
7	**eliminada**	boolean	NO	false	true si la columna fue eliminada de la tabla origen
8	**fecha_alta**	timestamp with time zone	NO	now()	Fecha de registro

aud_evento

Catálogo de eventos auditables (INSERT, UPDATE, DELETE)

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	smallint	NO	PK		Identificador del evento
2	**codigo**	character(1)	NO	UQ		Código corto: I, U o D
3	**nombre**	character varying(10)	NO	UQ		Nombre de la operación SQL

aud_configuracion

Eventos que deben auditarse por cada columna de cada tabla

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador de la configuración (referenciado por la bitácora)
2	**columna_id**	integer	NO	UQ, FK	autoaudit.aud_columna	Columna auditada (aud_columna)
3	**evento_id**	smallint	NO	UQ, FK	autoaudit.aud_evento	Evento auditado (aud_evento)
4	**activa**	boolean	NO		true	true si el evento está habilitado para la columna
5	**fecha_alta**	timestamp with time zone	NO		now()	Fecha de creación
6	**modificado**	timestamp with time zone	NO		now()	Fecha del último cambio de estado
7	**modificado_por**	name	NO		SESSION_USER	Administrador que hizo el último cambio

aud_usuario

Usuarios que han realizado cambios auditados

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción

1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador interno
2	**usuario_bd**	name	NO	UQ		Usuario de conexión a PostgreSQL (session_user)
3	**usuario_app**	character varying(120)	NO	UQ	''::character varying	Usuario de aplicación informado con SET autoaudit.usuario_app (vacío si no aplica)
4	**auditar**	boolean	NO		true	false para excluir de la auditoría (ej. procesos de carga masiva)
5	**primera_actividad**	timestamp with time zone	NO		now()	Primera vez que se registró actividad

aud_estacion

Estaciones de trabajo desde donde se realizaron cambios

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador interno
2	**ip**	character varying(80)	NO	UQ		IP origen almacenada según la política modo_ip (completa, enmascarada o hash)
3	**hostname**	character varying(255)	NO	UQ	''::character varying	Nombre de la estación informado por la aplicación con SET autoaudit.estacion
4	**aplicacion**	character varying(120)	NO	UQ	''::character varying	application_name de la conexión
5	**auditar**	boolean	NO		true	false para excluir la estación de la auditoría
6	**primera_actividad**	timestamp with time zone	NO		now()	Primera vez que se registró actividad

aud_bitacora

Bitácora centralizada: una fila por operación auditada

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	bigint	NO	PK	IDENTITY (ALWAYS)	Identificador del registro de bitácora
2	**fecha_hora**	timestamp with time zone	NO		clock_timestamp()	Fecha y hora exacta del cambio
3	**tabla_id**	integer	NO	FK	autoaudit.aud_tabla	Tabla origen (aud_tabla)

4	**evento_id**	smallint	NO	FK	autoaudit.aud_evento	Operación realizada (aud_evento)
5	**usuario_id**	integer	NO	FK	autoaudit.aud_usuario	Usuario responsable (aud_usuario)
6	**estacion_id**	integer	NO	FK	autoaudit.aud_estacion	Estación de trabajo de origen (aud_estacion)
7	**txid**	bigint	NO			Identificador de la transacción (agrupa cambios de una misma transacción)
8	**pk_registro**	jsonb	NO			Llave primaria del registro modificado (JSON)
9	**fila_anterior**	jsonb	SÍ			Fila completa antes del cambio (UPDATE/DELETE); preserva datos borrados
10	**fila_nueva**	jsonb	SÍ			Fila completa después del cambio (INSERT/UPDATE)

aud_bitacora_detalle

Valores antiguo y nuevo de cada columna afectada por una operación

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	bigint	NO	PK	IDENTITY (ALWAYS)	Identificador del detalle
2	**bitacora_id**	bigint	NO	FK	autoaudit.aud_bitacora	Registro de bitácora (cabecera)
3	**configuracion_id**	integer	NO	FK	autoaudit.aud_configuracion	Configuración (columna + evento) que originó el registro
4	**valor_anterior**	text	SÍ			Valor antes del cambio (NULL en INSERT)
5	**valor_nuevo**	text	SÍ			Valor después del cambio (NULL en DELETE)

aud_trigger

Historial y estado de los triggers generados (uno activo por tabla)

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	integer	NO	PK	IDENTITY (ALWAYS)	Identificador
2	**tabla_id**	integer	NO	FK	autoaudit.aud_tabla	Tabla origen que maneja el trigger (exactamente una)
3	**nombre_trigger**	name	NO			Nombre del trigger en pg_trigger

4	**nombre_funcion**	name	NO		Función PL/pgSQL generada que ejecuta el trigger
5	**trigger_oid**	oid	SÍ		OID del trigger en pg_trigger
6	**eventos**	character varying(20)	NO	''::character varying	Operaciones que cubre (ej. I,U,D)
7	**ddl**	text	NO		Código SQL generado
8	**hash_ddl**	character(32)	NO		MD5 del código, para detectar cambios
9	**estado**	character varying(20)	NO		ACTIVO, REEMPLAZADO, ELIMINADO, TABLA_ELIMINADA o ELIMINADO_EXTERNO
10	**generado**	timestamp with time zone	NO	now()	Fecha de generación
11	**generado_por**	name	NO	SESSION_USER	Administrador que lo generó

aud_error

Errores ocurridos al generar o ejecutar triggers de auditoría

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	bigint	NO	PK	IDENTITY (ALWAYS)	Identificador
2	**fecha**	timestamp with time zone	NO		clock_timestamp()	Momento del error
3	**usuario**	name	NO		SESSION_USER	Usuario que provocó el error
4	**contexto**	character varying(60)	NO			Dónde ocurrió: GENERACION, EJECUCION, VALIDACION, DDL
5	**tabla_id**	integer	SÍ	FK	autoaudit.aud_tabla	Tabla relacionada (si aplica)
6	**sqlstate**	character(5)	SÍ			Código SQLSTATE de PostgreSQL
7	**mensaje**	text	NO			Mensaje de error
8	**detalle**	text	SÍ			Detalle, pista o contexto adicional

aud_cambio_config

Auditoría de la propia configuración de AutoAudit

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**id**	bigint	NO	PK	IDENTITY (ALWAYS)	Identificador

2	**fecha**	timestamp with time zone	NO		clock_timestamp()	Fecha del cambio
3	**usuario**	name	NO		SESSION_USER	Administrador que realizó el cambio
4	**accion**	character varying(40)	NO			Acción: CONFIGURAR, DESACTIVAR_TABLA, GENERAR, PARAMETRO, DDL_EXTERNO...
5	**tabla_id**	integer	SÍ	FK	autoaudit.aud_tabla	Tabla afectada (si aplica)
6	**detalle**	jsonb	NO		'{}'::jsonb	Detalle del cambio en JSON

aud_parametro

Parámetros de funcionamiento de AutoAudit

#	Columna	Tipo	Nulo	Llave	Referencia / defecto	Descripción
1	**clave**	character varying(60)	NO	PK		Nombre del parámetro
2	**valor**	text	NO			Valor actual del parámetro
3	**descripcion**	text	NO			Explicación del parámetro y valores permitidos
4	**modificado**	timestamp with time zone	NO		now()	Fecha y hora de la última modificación
5	**modificado_por**	name	NO		SESSION_USER	Usuario de base de datos que lo modificó

5. Funcionamiento interno

5.1 Consultas al diccionario de datos del DBMS

El módulo de configuración solo ofrece objetos que existen. Las funciones consultan `pg_class`, `pg_namespace`, `pg_attribute`, `pg_attrdef` y `pg_constraint`:

```
-- Tablas disponibles (excluye esquemas del sistema y particiones)
SELECT n.nspname, c.relname, c.reltuples, EXISTS (SELECT 1 FROM pg_constraint k
       WHERE k.conrelid = c.oid AND k.contype = 'p') AS tiene_pk
  FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
 WHERE c.relkind IN ('r','p') AND NOT c.relispartition
   AND n.nspname NOT IN ('pg_catalog','information_schema','autoaudit');

-- Columnas de una tabla con su tipo y si forman la llave primaria
SELECT a.attnum, a.attname, format_type(a.atttypid, a.atttypmod), a.attnotnull
  FROM pg_attribute a WHERE a.attrelid = 'demo.clientes'::regclass
   AND a.attnum > 0 AND NOT a.attisdropped;
```

Antes de activar, `fn_validar_config` comprueba que la tabla y cada columna existan, advierte si la tabla no tiene llave primaria y sincroniza nombres por OID y `attnum`.

5.2 Generación dinámica de triggers

`_fn_generar_trigger(tabla_id)` lee la configuración activa y construye con `format('%I')`, a prueba de inyección SQL:

- **Una función PL/pgSQL por tabla** (`autoaudit.aa_fn_t<id>_<esquema>_<tabla>`) con el código "desenrollado": cada columna configurada aparece explícita, sin bucles ni reflexión en tiempo de ejecución.
- **Un único trigger por tabla origen**: `CREATE OR REPLACE TRIGGER aa_auditoria AFTER INSERT OR UPDATE OF <columnas> OR DELETE … FOR EACH ROW`. Solo incluye las operaciones habilitadas, y `UPDATE OF` limita el disparo a las columnas auditadas.
- En UPDATE solo se registra si alguna columna auditada cambió realmente (`IS DISTINCT FROM`); los tipos sin operador de igualdad (json, xml, arreglos) se comparan como texto.
- El código y su hash MD5 se guardan en `aud_trigger`. Si el código no cambió, no se recrea (operación idempotente). Si ya no hay columnas activas, se eliminan el trigger y la función.

Ejemplo real generado para `demo.productos` (fragmento):

```
CREATE OR REPLACE TRIGGER aa_auditoria
AFTER INSERT OR UPDATE OF precio, existencia, activo, atributos OR DELETE ON demo.productos
FOR EACH ROW EXECUTE FUNCTION autoaudit.aa_fn_t2_demo_productos();

ELSIF TG_OP = 'UPDATE' THEN
    IF NOT (NEW.precio IS DISTINCT FROM OLD.precio OR NEW.existencia IS DISTINCT FROM
OLD.existencia ...) THEN
        RETURN NULL;  -- ninguna columna auditada cambió realmente
    END IF;
    INSERT INTO autoaudit.aud_bitacora (...) VALUES (v_fecha, 2, 2, v_ctx[1], v_ctx[2], ...,
to_jsonb(OLD), to_jsonb(NEW))
    RETURNING id INTO v_id;
    INSERT INTO autoaudit.aud_bitacora_detalle (bitacora_id, configuracion_id, valor_anterior,
valor_nuevo)
    SELECT v_id, v.cfg, v.anterior, v.nuevo FROM (VALUES (15, OLD.precio::text, NEW.precio::text,
...)) ...
```

5.3 Bitácora

- **Cabecera** (`aud_bitacora`): tabla, operación, usuario, estación, identificador de transacción, fecha y hora (`clock_timestamp()`), llave primaria del registro y la **fila completa** antes y después del cambio (`jsonb`).
- **Detalle** (`aud_bitacora_detalle`): una fila por columna afectada con su valor anterior y nuevo y la configuración que la originó.
- **Contexto**: `_fn_contexto()` obtiene `session_user` , el usuario de aplicación (`SET autoaudit.usuario_app`), la estación (`SET autoaudit.estacion`), `application_name` e `inet_client_addr()` . Usa una caché por sesión que PostgreSQL revierte si la transacción se revierte.

5.4 Gestión de dependencias entre triggers y tablas origen

Evento DDL	Reacción automática (event triggers)
`DROP TABLE` auditada	La configuración se marca como eliminada, el trigger pasa a `TABLA_ELIMINADA` y se borra la función. La bitácora se conserva.
`DROP COLUMN` auditada	Sin `CASCADE` , PostgreSQL lo impide (dependencia de `UPDATE OF`). Con `CASCADE` , la columna se desactiva y el trigger se regenera solo.
`RENAME` de tabla o columna	Los nombres se sincronizan por OID / `attnum` y el trigger se regenera.
`DROP TRIGGER` o `DISABLE TRIGGER` manual	Se registra quién lo hizo en `aud_cambio_config` y `aud_error` ; la vista `v_estado_triggers` lo muestra como pendiente o deshabilitado.

5.5 Consultas parametrizadas para la consulta histórica

Función	Uso
`fn_cambios_por_usuario(usuario, desde, hasta, esquema, tabla)`	Historial completo de un usuario de BD o de aplicación.

`fn_cambios_por_columna(esquema, tabla, columna, desde, hasta, operación)`	Todos los cambios de una columna; sobrevive a renombres.
`fn_historial_registro(esquema, tabla, pk)`	Cronología de un registro por su llave primaria.
`fn_estado_en_fecha(esquema, tabla, pk, fecha)`	Reconstruye cómo estaba un registro en una fecha.
`fn_resumen`, `fn_actividad_diaria`	Indicadores y gráfica del panel.

Desde Python siempre se llaman con parámetros (`%s` de psycopg), nunca concatenando texto:

```
conn.execute("SELECT * FROM autoaudit.fn_cambios_por_columna(%s, %s, %s, %s, %s, %s)",
             (esquema, tabla, columna, desde, hasta, operacion))
```

6. Manual de configuración y activación de auditoría por tabla

6.1 Ingreso

Abra la aplicación (por defecto `http://localhost:8000`) e ingrese con un usuario de PostgreSQL que tenga el rol `autoaudit_admin` (configura) o `autoaudit_lector` (solo consulta).

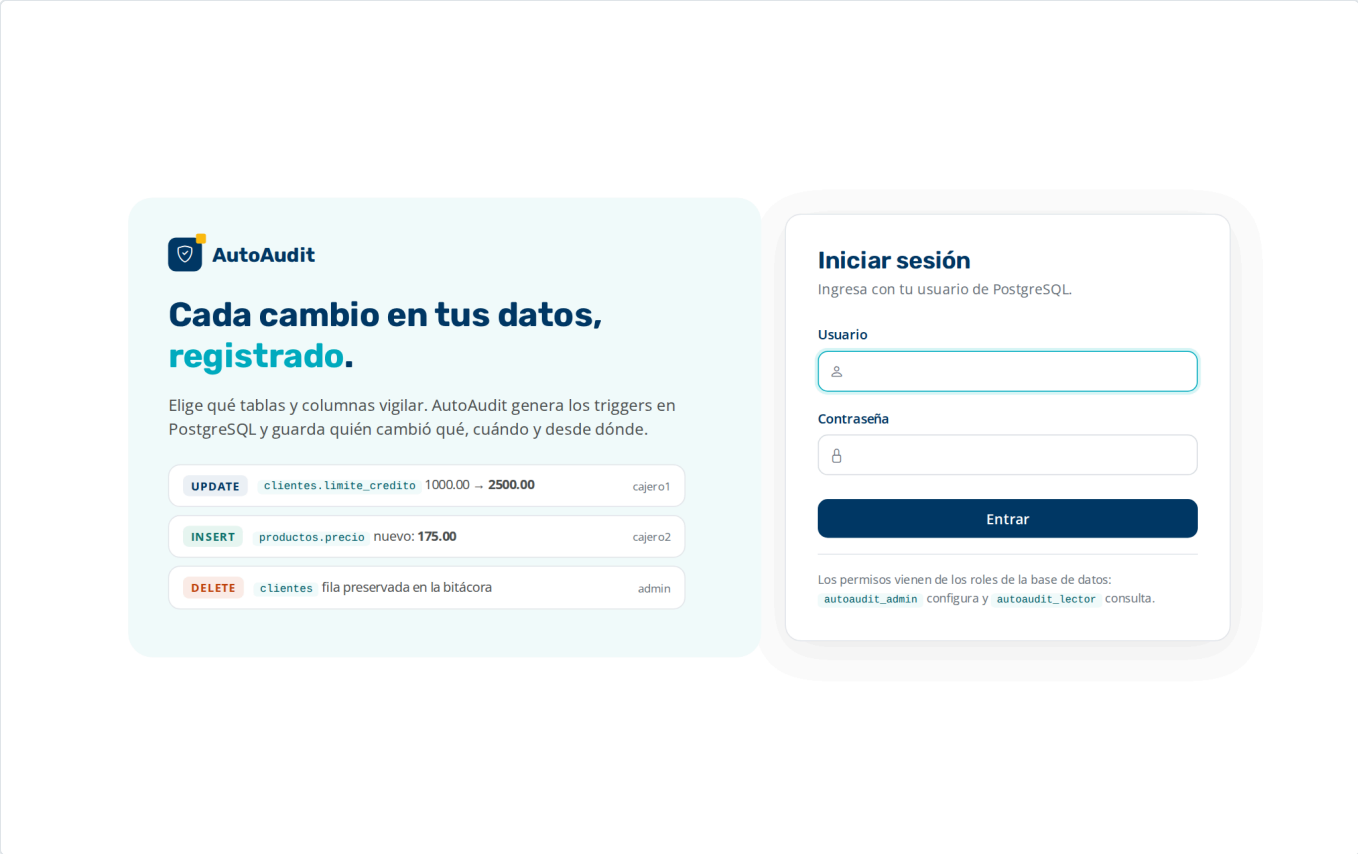


Figura 2. Inicio de sesión.

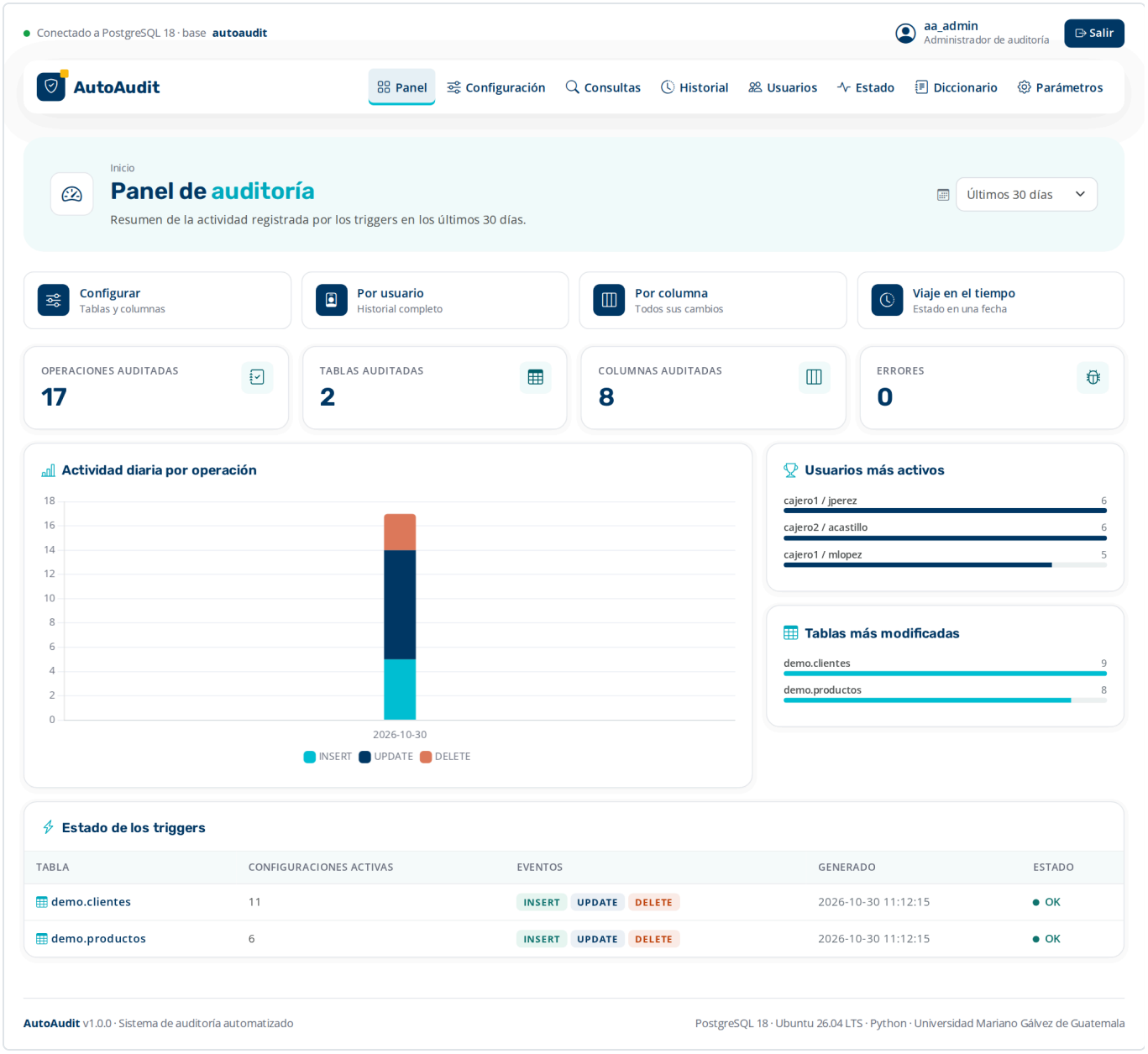


Figura 3. Panel principal con indicadores, actividad y estado de los triggers.

6.2 Seleccionar la tabla

En **Configuración** se listan las tablas existentes según el diccionario de datos, con su llave primaria y el estado de su auditoría. Presione **Configurar** en la tabla deseada.

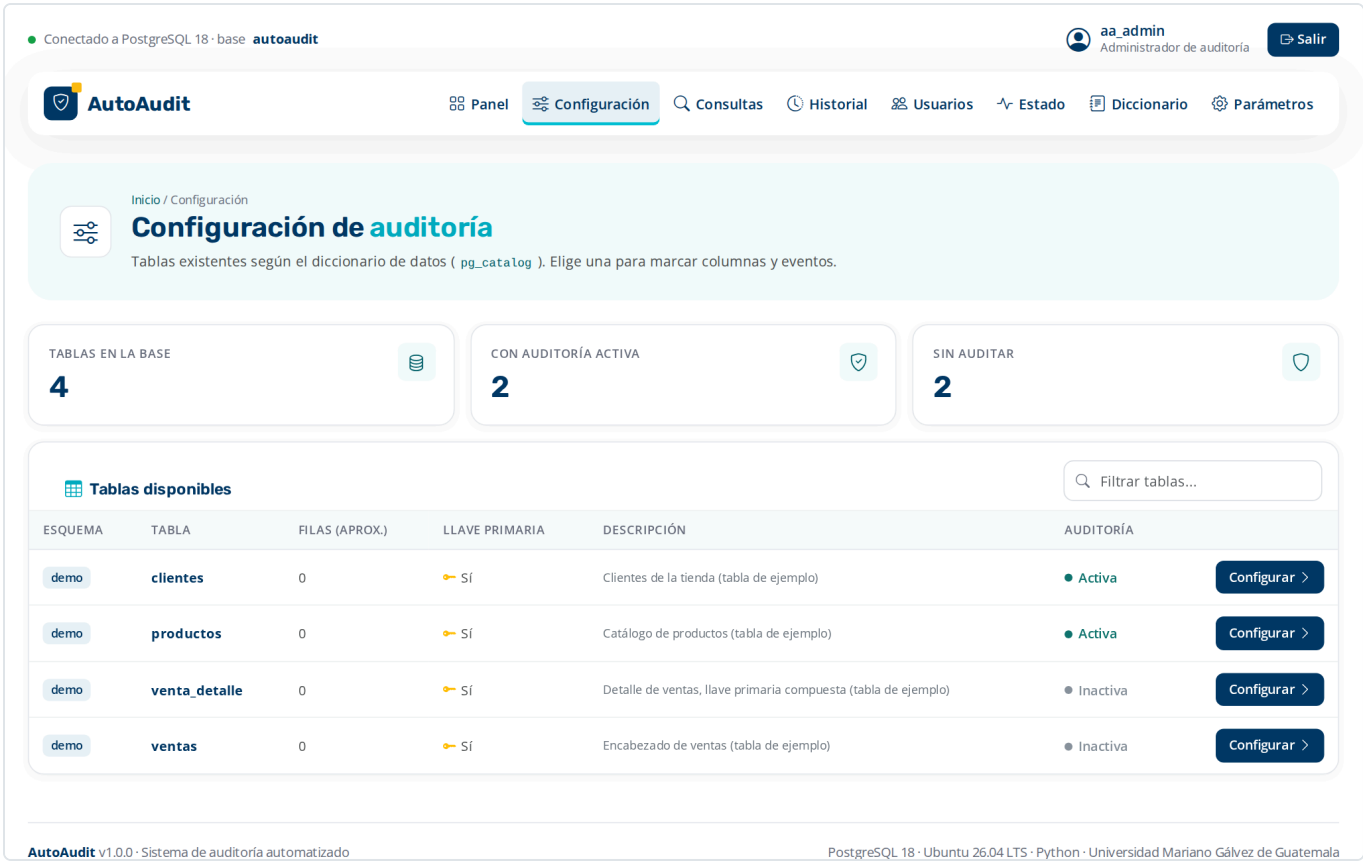


Figura 4. Tablas disponibles para auditar.

6.3 Elegir columnas y eventos

1. Marque, por cada columna, los eventos a auditar: **INSERT**, **UPDATE** y/o **DELETE**. La casilla del encabezado marca la columna completa.
2. Presione **Vista previa del SQL** para ver el código exacto que se generará, sin guardar nada.
3. Presione **Guardar y generar trigger**. El sistema valida, guarda la configuración y crea o actualiza el trigger. El resultado se muestra en un aviso y en el historial de triggers.
4. Para dejar de auditar la tabla use **Desactivar auditoría**: se elimina el trigger y la bitácora histórica se conserva.

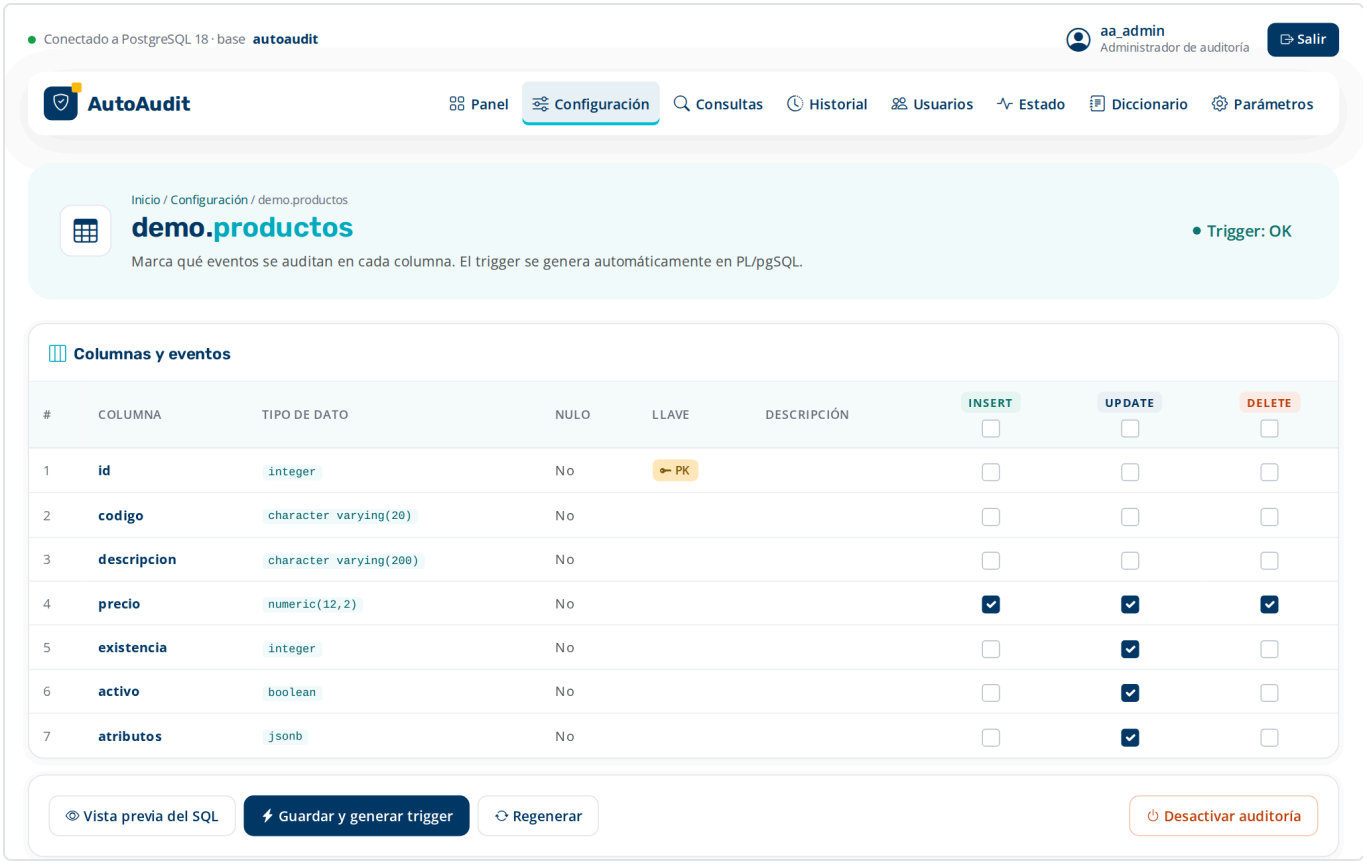


Figura 5. Selección granular de columnas y eventos.

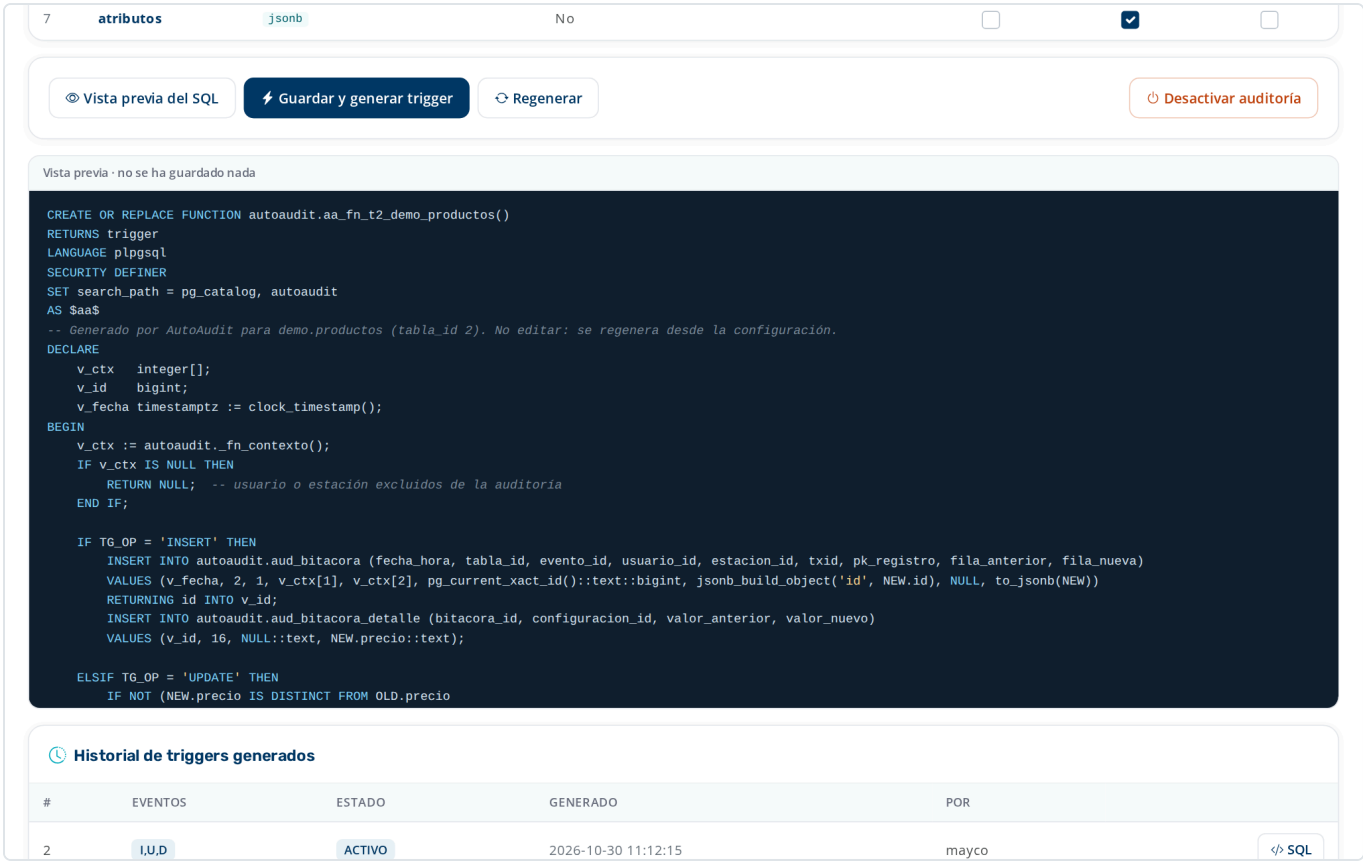


Figura 6. Vista previa del SQL generado.

6.4 Equivalente por línea de comandos y SQL

```
# CLI
python cli.py tablas
python cli.py configurar demo clientes nombre=IUD correo=IUD limite_credito=IUD nit=ID
python cli.py previsualizar demo ventas total=U estado=UD
python cli.py estado

-- SQL (como administrador de auditoría)
SELECT autoaudit.fn_configurar_tabla('demo', 'clientes',
       '{"nombre": ["I","U","D"], "correo": ["U"]}');
SELECT autoaudit.fn_configurar_columna('demo', 'clientes', 'telefono', false, true, false);
SELECT autoaudit.fn_desactivar_tabla('demo', 'clientes');
```

6.5 Verificar el estado

La pantalla **Estado** (vista `v_estado_triggers`) compara la configuración con los triggers que realmente existen: `OK`, `PENDIENTE`, `TRIGGER_FALTANTE`, `DESHABILITADO` o `DESACTUALIZADO`. También muestra los errores y el historial de cambios de configuración.

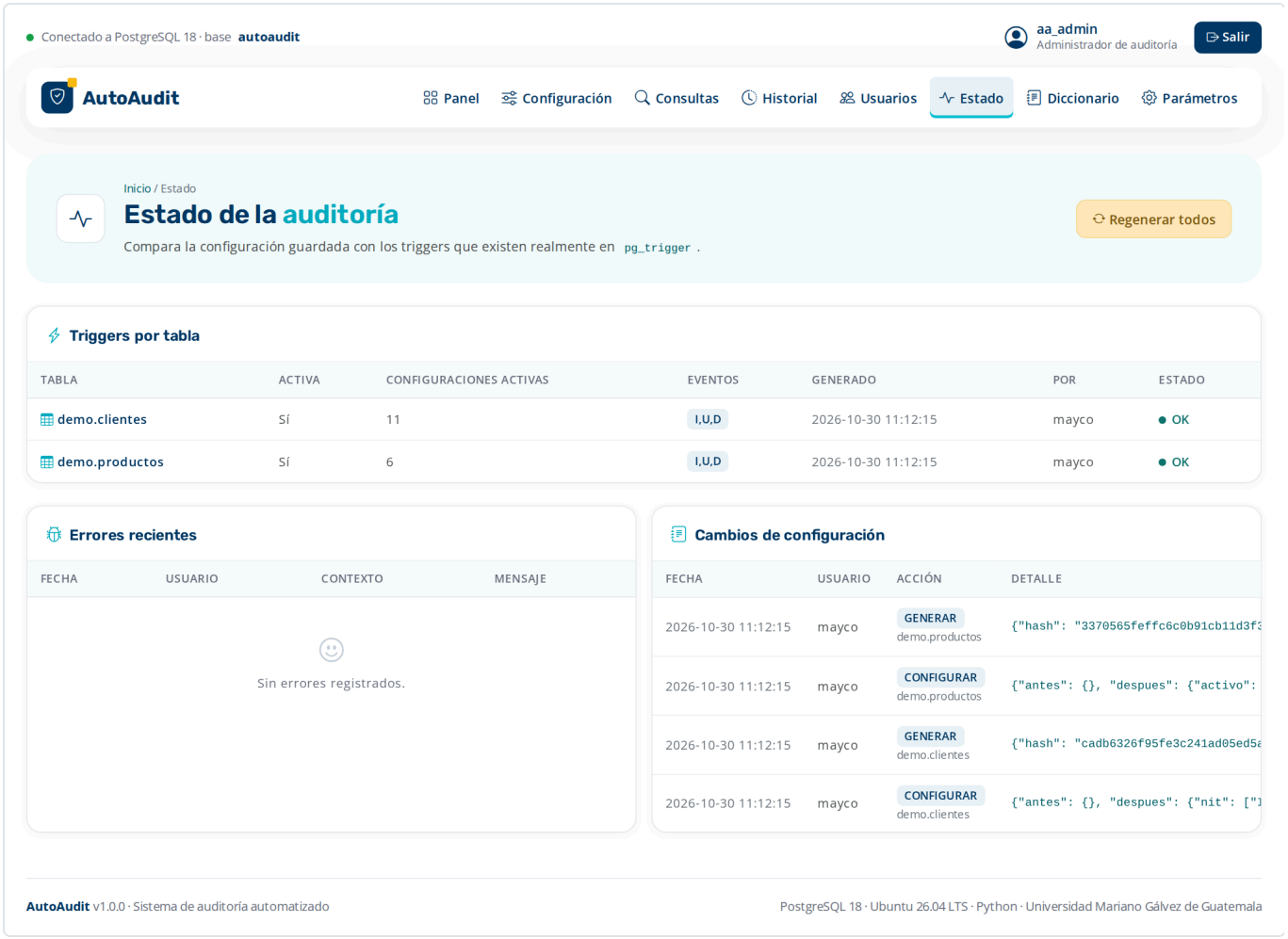


Figura 7. Estado de la auditoría, errores y cambios de configuración.

6.6 Consultar el historial

En **Consultas** se busca por usuario o por columna, con filtro de fechas y operación, en vista de tabla o de línea de tiempo. Los botones CSV, XLSX, PDF y JSON exportan el resultado completo.

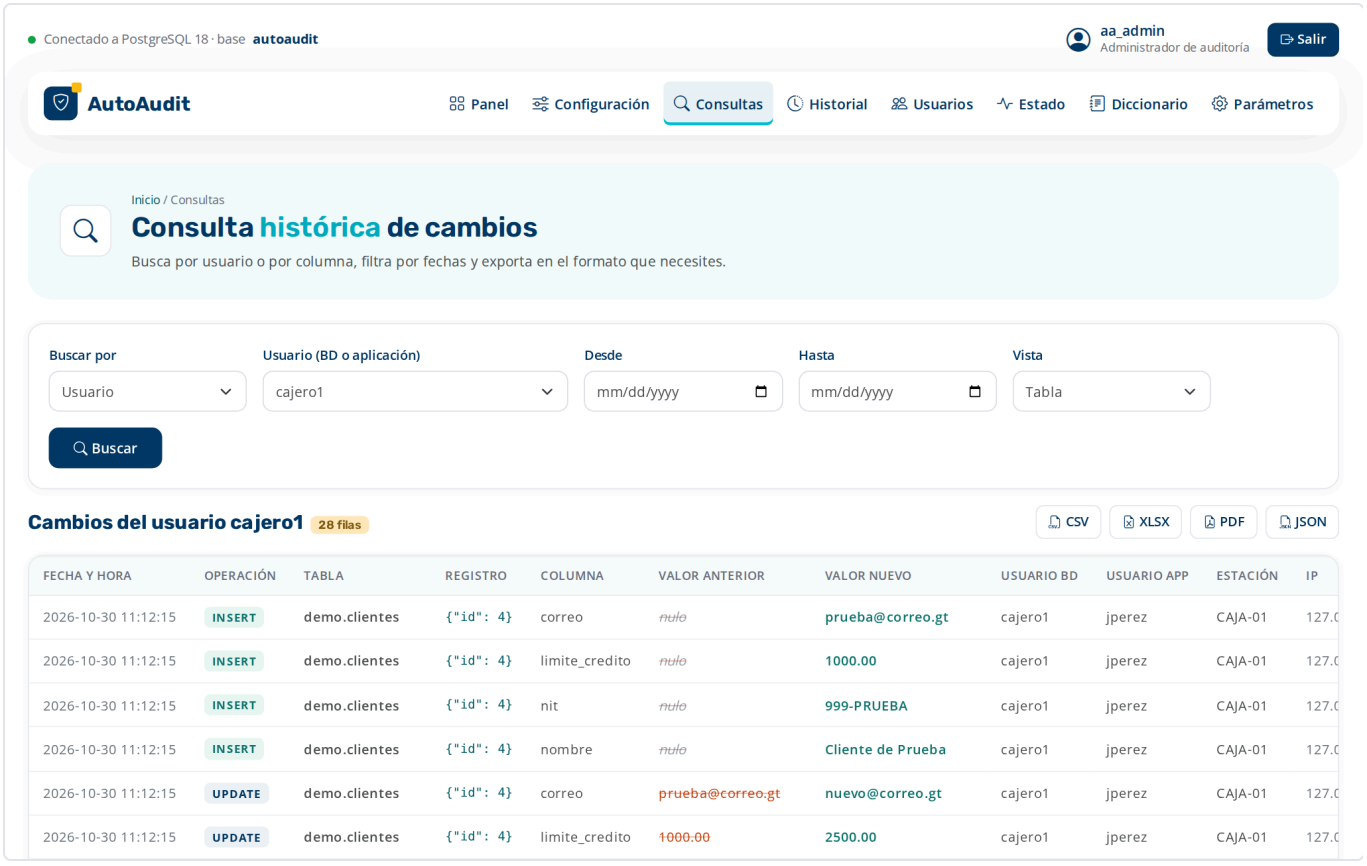


Figura 8. Consulta por usuario.

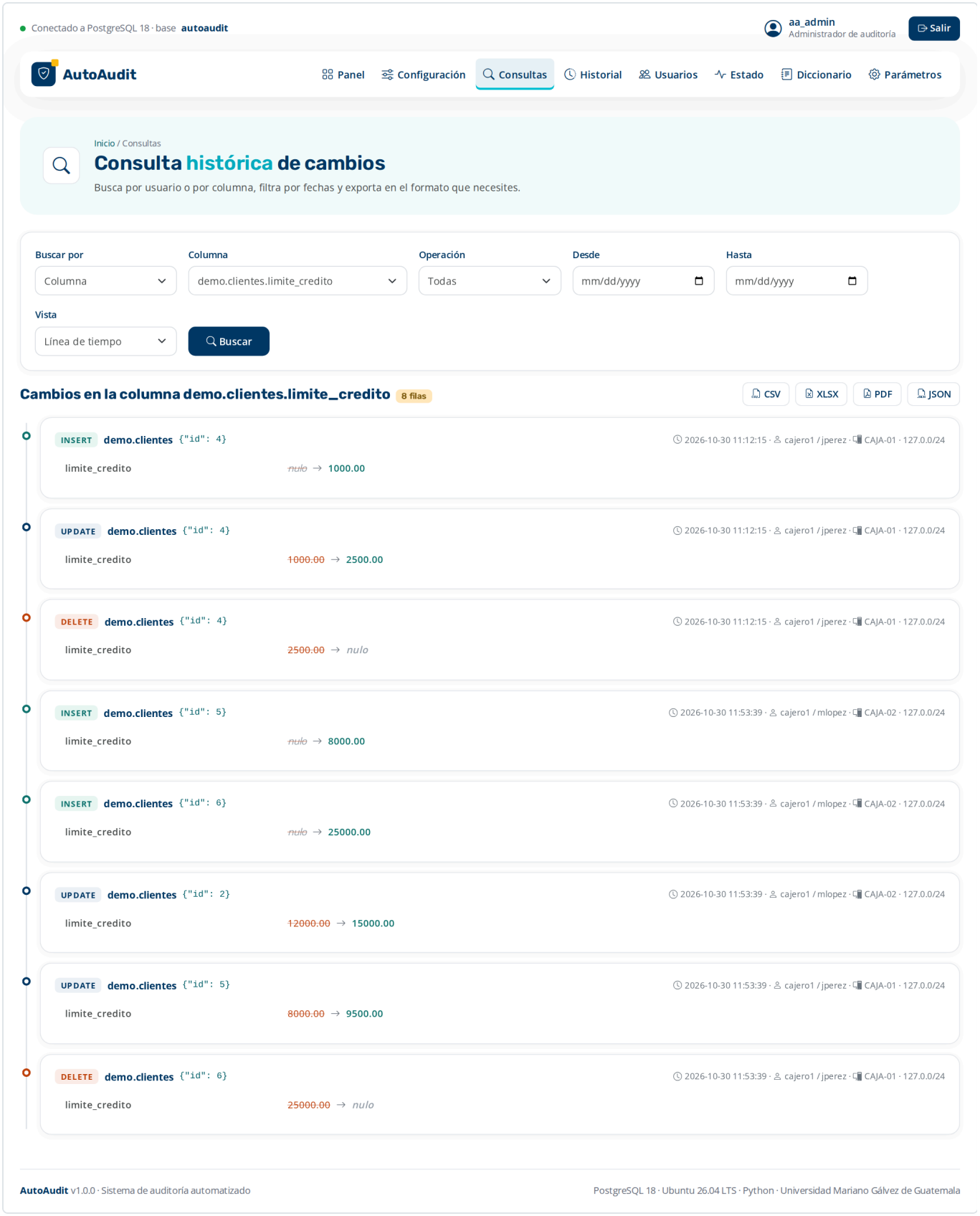


Figura 9. Consulta por columna en línea de tiempo.

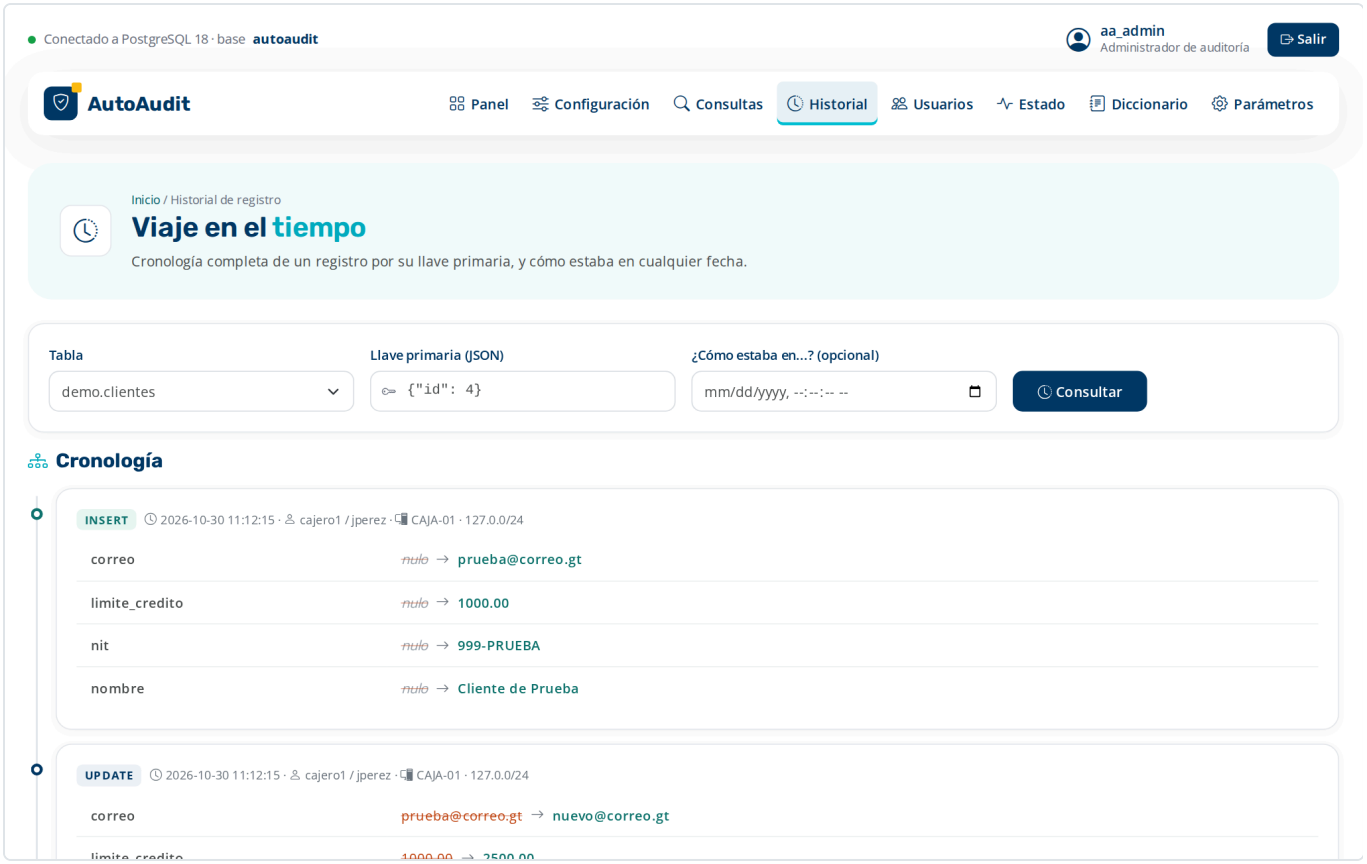


Figura 10. Historial de un registro y su estado en una fecha.

6.7 Usuarios, estaciones y parámetros

Los usuarios y estaciones se registran solos la primera vez que hacen un cambio. Un administrador puede excluir, por ejemplo, el usuario de un proceso de carga masiva. En **Parámetros** se elige cómo se guarda la IP (completa, enmascarada o hash) y el modo de error (estricto o tolerante).

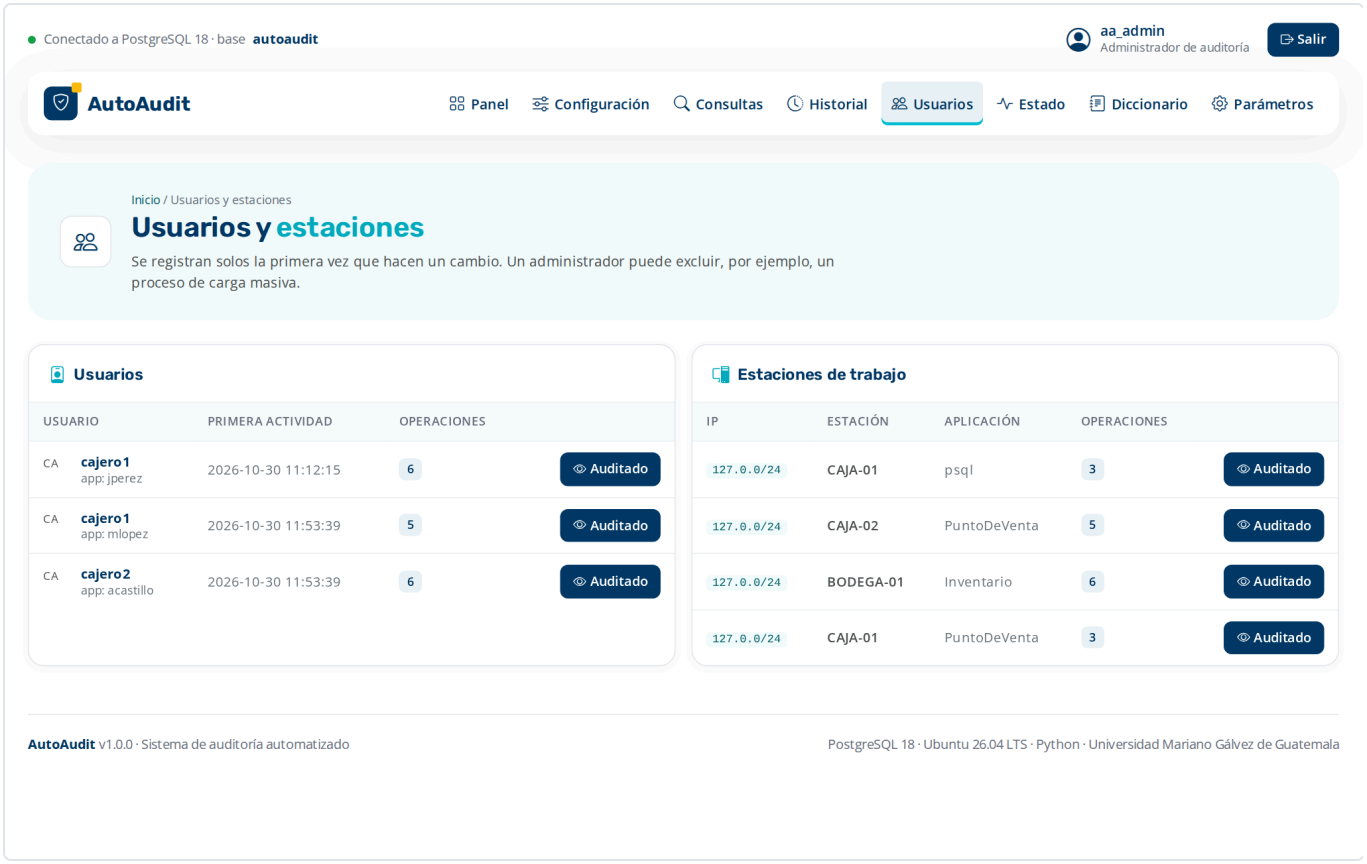


Figura 11. Usuarios y estaciones auditadas.

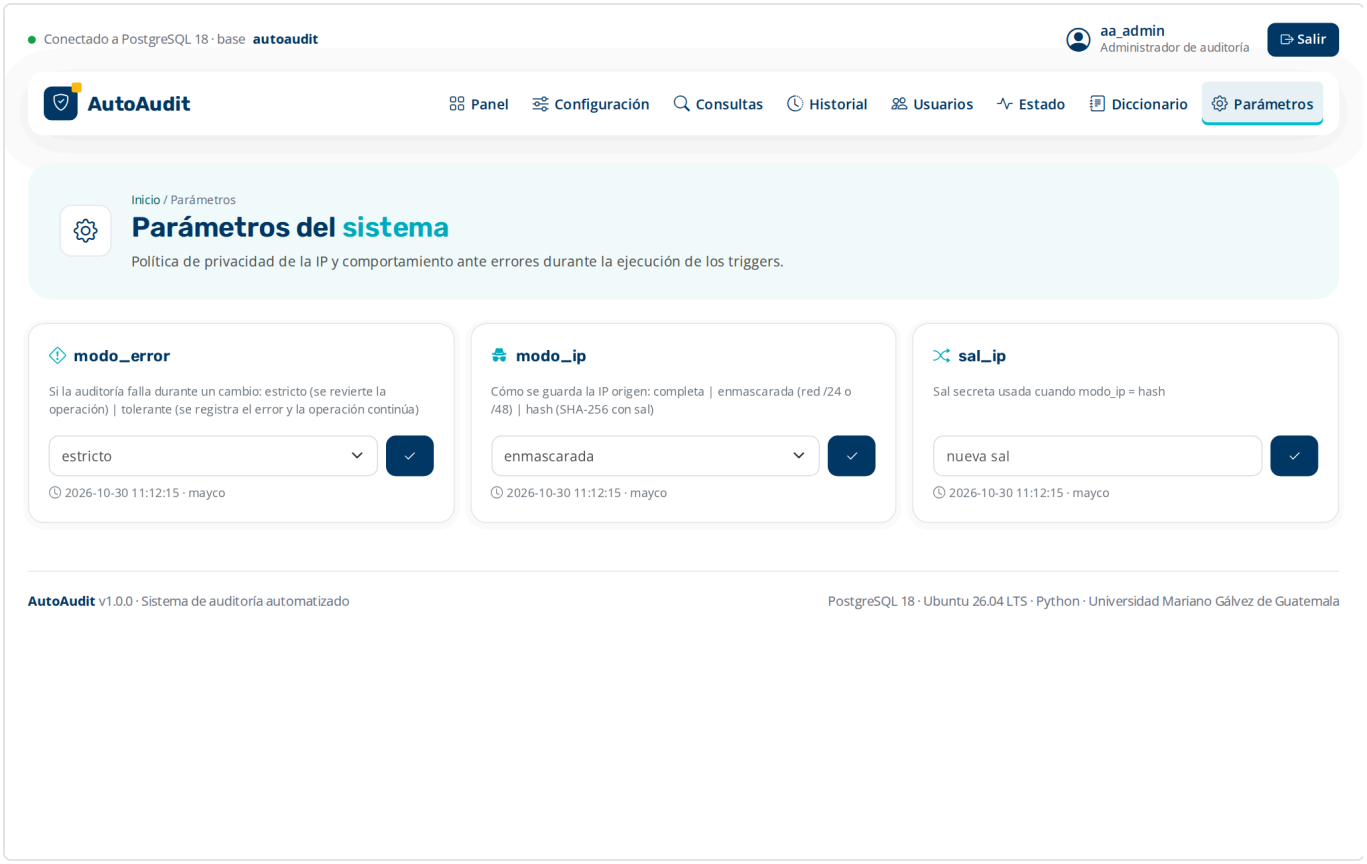


Figura 12. Parámetros del sistema.

7. Procedimiento de instalación y despliegue de triggers

7.1 Requisitos

- Ubuntu 26.04 LTS (servidor o WSL2).
- PostgreSQL 18 y acceso de superusuario para la instalación (los event triggers lo requieren).
- Python 3.12 o superior.

7.2 Instalación automática

```
git clone https://github.com/Gaston1x/autoaudit.git
cd autoaudit
sudo bash scripts/install.sh            # con datos de ejemplo
sudo DEMO=0 bash scripts/install.sh     # sin datos de ejemplo
```

El script instala PostgreSQL 18 (repositorio de Ubuntu o, si no está, el oficial PGDG), crea la base `autoaudit`, ejecuta `sql/instalar.sql`, crea el entorno virtual de Python e instala las dependencias.

7.3 Instalación manual paso a paso

```
sudo apt-get update
sudo apt-get install -y postgresql-18 postgresql-client-18 python3-venv
sudo -u postgres createdb autoaudit
sudo -u postgres psql -d autoaudit -f sql/instalar.sql         # -v demo=0 para omitir el ejemplo
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cp .env.example .env
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Script	Contenido
`00_roles.sql`	Roles `autoaudit_owner`, `autoaudit_admin` y `autoaudit_lector`.
`01_esquema_tablas.sql`	Esquema y tablas del sistema con comentarios.
`02_diccionario.sql`	Consultas al diccionario de datos.
`03_configuracion.sql`	Validación y persistencia de la configuración.
`04_generador_triggers.sql`	Generador dinámico de triggers y contexto.
`05_event_triggers.sql`	Gestión de dependencias DDL.
`06_consultas.sql`	Vistas y consultas parametrizadas.
`07_seguridad.sql`	Permisos y bitácora inmutable.

`90_demo_datos.sql`	Esquema de ejemplo y usuarios de prueba.

7.4 Preparar un esquema de negocio propio

Para crear y eliminar triggers PostgreSQL exige ser dueño de la tabla. Se delega el rol dueño del esquema de negocio al dueño de AutoAudit y se crean los usuarios de auditoría:

```
GRANT dueno_de_mis_tablas TO autoaudit_owner;
CREATE ROLE auditor1 LOGIN PASSWORD '...';   GRANT autoaudit_admin  TO auditor1;
CREATE ROLE consulta1 LOGIN PASSWORD '...';  GRANT autoaudit_lector TO consulta1;
```

7.5 Despliegue de los triggers

1. Configure cada tabla (sección 6). Al guardar, el trigger se despliega en la misma transacción.
2. Para redesplegar todos los triggers (por ejemplo, tras restaurar un respaldo): botón **Regenerar todos** en Estado, o `SELECT * FROM autoaudit.fn_regenerar_todos(true);`
3. Verifique con `SELECT * FROM autoaudit.v_estado_triggers;` que todas las tablas estén en `OK`.
4. Las aplicaciones que usen un único usuario de conexión pueden identificar al usuario real con `SET autoaudit.usuario_app = 'jperez'` y la estación con `SET autoaudit.estacion = 'CAJA-01'`.

8. Seguridad y privacidad

Requisito	Implementación
Solo administradores configuran	Las funciones de configuración solo tienen `EXECUTE` para `autoaudit_admin` y, además, verifican `pg_has_role(session_user, 'autoaudit_admin')` (defensa en profundidad). La web también lo exige.
Sin escritura directa en la bitácora	Las tablas pertenecen a `autoaudit_owner` (NOLOGIN); nadie más tiene INSERT, UPDATE, DELETE ni TRUNCATE. Solo las funciones de trigger (`SECURITY DEFINER` con `search_path` fijo) escriben. Un trigger `BEFORE UPDATE/DELETE/TRUNCATE` hace la bitácora inmutable incluso para el superusuario.
Privacidad de la IP	Parámetro `modo_ip` : completa, **enmascarada** (por defecto, red /24 o /48) o hash SHA-256 con sal secreta. El rol lector siempre ve la IP enmascarada y no puede leer la columna ni la sal.
Inyección SQL	`format('%I', …)` para identificadores en PL/pgSQL y parámetros en psycopg.
Credenciales	Cada persona entra con su propio usuario de PostgreSQL; la sesión web guarda solo un token aleatorio en una cookie `HttpOnly` .
Funciones generadas	Sin `EXECUTE` para `PUBLIC` .

9. Manejo de errores

- **Validación previa:** tabla o columna inexistente, evento no válido o esquema del sistema se rechazan con un mensaje claro y un código SQLSTATE específico.
- **Errores de generación:** la generación corre en un subbloque; si falla (por ejemplo, falta de privilegios sobre la tabla) se revierte por completo, sin dejar triggers a medias, se registra en `aud_error` con SQLSTATE, mensaje, pista y SQL, y se informa al usuario.
- **Errores de ejecución** (parámetro `modo_error`):
  - *estricto* (por defecto): si la auditoría falla, la operación de negocio se revierte; ningún cambio queda sin auditar.
  - *tolerante*: el error se guarda en `aud_error` y la operación continúa.
- **Errores de la aplicación:** se muestran como avisos y se registran con `fn_registrar_error`.

10. Pruebas de validación

10.1 Pruebas funcionales

Se ejecutan con `pytest` sobre una base de datos creada desde cero en cada corrida (`autoaudit_test`). Resultado: **81 de 81 pruebas aprobadas** en 6.8 s.

Archivo	Pruebas	Aprobadas
`tests/test_configuracion.py`	19	**19**
`tests/test_consultas.py`	7	**7**
`tests/test_operaciones.py`	12	**12**
`tests/test_seguridad.py`	23	**23**
`tests/test_web.py`	20	**20**

Casos requeridos por el enunciado

Caso	Prueba
INSERT con registro correcto en bitácora	`test_insert_registra_todos_los_campos` : cabecera, fila completa, valores por columna, usuario, estación, aplicación, IP y fecha.
UPDATE con valor antiguo y nuevo	`test_update_captura_valor_antiguo_y_nuevo` , `test_update_sin_cambios_reales_no_se_registra` .
DELETE con preservación de datos	`test_delete_preserva_los_datos` : la fila completa sigue en la bitácora tras borrarla.
Rendimiento con volumen creciente	`pruebas_rendimiento.py` (sección 10.2).

Detalle de todas las pruebas

Archivo	Prueba	Resultado
test_configuracion	test_tabla_inexistente	**OK**
test_configuracion	test_columna_inexistente	**OK**
test_configuracion	test_evento_invalido	**OK**
test_configuracion	test_no_se_auditan_tablas_del_sistema	**OK**
test_configuracion	test_diccionario_lista_tablas_y_columnas	**OK**
test_configuracion	test_desactivar_evento_lo_quita_del_trigger	**OK**
test_configuracion	test_configuracion_refleja_estado_actual	**OK**
test_configuracion	test_regenerar_sin_cambios_es_idempotente	**OK**

test_configuracion	test_vista_previa_no_guarda	**OK**
test_configuracion	test_historial_de_cambios_de_configuracion	**OK**
test_configuracion	test_drop_column_regenera_trigger	**OK**
test_configuracion	test_drop_column_sin_cascade_esta_protegido	**OK**
test_configuracion	test_rename_column_y_tabla	**OK**
test_configuracion	test_drop_table_conserva_bitacora	**OK**
test_configuracion	test_drop_trigger_manual_queda_registrado	**OK**
test_configuracion	test_trigger_deshabilitado_se_detecta	**OK**
test_configuracion	test_error_de_generacion_se_registra_y_no_deja_triggers_a_medias	**OK**
test_configuracion	test_modo_tolerante_registra_error_de_ejecucion	**OK**
test_configuracion	test_modo_estricto_revierte_la_operacion	**OK**
test_consultas	test_consulta_por_usuario_historial_completo	**OK**
test_consultas	test_consulta_por_columna_todos_los_cambios	**OK**
test_consultas	test_filtro_por_fechas	**OK**
test_consultas	test_consulta_por_columna_sobrevive_renombre	**OK**
test_consultas	test_historial_y_estado_en_fecha	**OK**
test_consultas	test_lector_ve_ip_enmascarada	**OK**
test_consultas	test_modo_ip_hash	**OK**
test_operaciones	test_trigger_generado_es_valido	**OK**
test_operaciones	test_insert_registra_todos_los_campos	**OK**
test_operaciones	test_update_captura_valor_antiguo_y_nuevo	**OK**
test_operaciones	test_update_sin_cambios_reales_no_se_registra	**OK**
test_operaciones	test_delete_preserva_los_datos	**OK**
test_operaciones	test_solo_columnas_y_eventos_configurados	**OK**
test_operaciones	test_tipos_sin_operador_de_igualdad	**OK**
test_operaciones	test_llave_primaria_compuesta	**OK**
test_operaciones	test_cambios_en_una_transaccion_comparten_txid	**OK**
test_operaciones	test_rollback_no_deja_rastro	**OK**
test_operaciones	test_cache_de_contexto_no_sobrevive_a_un_rollback	**OK**
test_operaciones	test_usuario_excluido_no_se_audita	**OK**
test_seguridad	test_solo_administradores_configuran[cajero1]	**OK**
test_seguridad	test_solo_administradores_configuran[lector]	**OK**

test_seguridad	test_administrador_si_configura	**OK**
test_seguridad	test_chequeo_interno_de_administrador	**OK**
test_seguridad	test_sin_escritura_directa[sql0-admin]	**OK**
test_seguridad	test_sin_escritura_directa[sql0-lector]	**OK**
test_seguridad	test_sin_escritura_directa[sql0-cajero1]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_bitacora_detalle SET valor_nuevo = 'x'-admin]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_bitacora_detalle SET valor_nuevo = 'x'-lector]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_bitacora_detalle SET valor_nuevo = 'x'-cajero1]	**OK**
test_seguridad	test_sin_escritura_directa[DELETE FROM autoaudit.aud_bitacora-admin]	**OK**
test_seguridad	test_sin_escritura_directa[DELETE FROM autoaudit.aud_bitacora-lector]	**OK**
test_seguridad	test_sin_escritura_directa[DELETE FROM autoaudit.aud_bitacora-cajero1]	**OK**
test_seguridad	test_sin_escritura_directa[TRUNCATE autoaudit.aud_bitacora-admin]	**OK**
test_seguridad	test_sin_escritura_directa[TRUNCATE autoaudit.aud_bitacora-lector]	**OK**
test_seguridad	test_sin_escritura_directa[TRUNCATE autoaudit.aud_bitacora-cajero1]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_configuracion SET activa = false-admin]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_configuracion SET activa = false-lector]	**OK**
test_seguridad	test_sin_escritura_directa[UPDATE autoaudit.aud_configuracion SET activa = false-cajero1]	**OK**
test_seguridad	test_bitacora_inmutable_incluso_para_superusuario	**OK**
test_seguridad	test_usuario_de_negocio_no_lee_la_bitacora	**OK**
test_seguridad	test_lector_no_ve_ip_ni_sal_directamente	**OK**
test_seguridad	test_funciones_generadas_no_son_publicas	**OK**
test_web	test_login_incorrecto	**OK**
test_web	test_usuario_sin_rol_de_auditoria_no_entra	**OK**
test_web	test_paginas_requieren_sesion	**OK**
test_web	test_paginas_cargan[/]	**OK**
test_web	test_paginas_cargan[/tablas]	**OK**
test_web	test_paginas_cargan[/tablas/demo/clientes]	**OK**
test_web	test_paginas_cargan[/consultas]	**OK**

test_web	test_paginas_cargan[/registro]	**OK**
test_web	test_paginas_cargan[/usuarios]	**OK**
test_web	test_paginas_cargan[/estado]	**OK**
test_web	test_paginas_cargan[/diccionario]	**OK**
test_web	test_paginas_cargan[/diccionario?esquema=demo]	**OK**
test_web	test_lector_no_puede_configurar_desde_la_web	**OK**
test_web	test_admin_configura_desde_la_web	**OK**
test_web	test_exportar_desde_la_web[csv-text/csv]	**OK**
test_web	test_exportar_desde_la_web[xlsx-spreadsheetml]	**OK**
test_web	test_exportar_desde_la_web[pdf-application/pdf]	**OK**
test_web	test_exportar_desde_la_web[json-application/json]	**OK**
test_web	test_exportar_csv_con_tildes	**OK**
test_web	test_exportar_json_y_formatos_binarios	**OK**

10.2 Pruebas de rendimiento

Se comparan dos tablas idénticas, una sin auditoría y otra con tres columnas auditadas en INSERT, UPDATE y DELETE, con volúmenes de 1,000 a 100,000 filas por operación, mientras la bitácora crece hasta más de un millón de filas.

Filas	INSERT sin / con (ms)	UPDATE sin / con (ms)	DELETE sin / con (ms)	Usuario excluido INSERT (ms)	Filas en bitácora
1,000	5.7 / 130.2	8.6 / 111.0	3.8 / 119.4	10.8	12,000
5,000	15.8 / 586.3	21.9 / 537.7	6.5 / 637.4	36.9	52,000
10,000	26.5 / 1212.5	41.2 / 1119.2	8.3 / 1207.7	73.2	132,000
50,000	123.4 / 6045.4	221.4 / 5508.6	25.9 / 5771.0	363.0	532,000
100,000	283.8 / 12113.4	408.1 / 10791.3	52.1 / 11972.4	703.0	1,332,000

Filas en bitácora	Historial de un registro (ms)	Por usuario (ms)	Por columna y operación (ms)
12,000	5.73	3.37	0.61
52,000	7.64	1.19	0.3
132,000	14.14	2.27	0.39
532,000	7.98	0.71	0.43
1,332,000	4.18	0.55	0.42

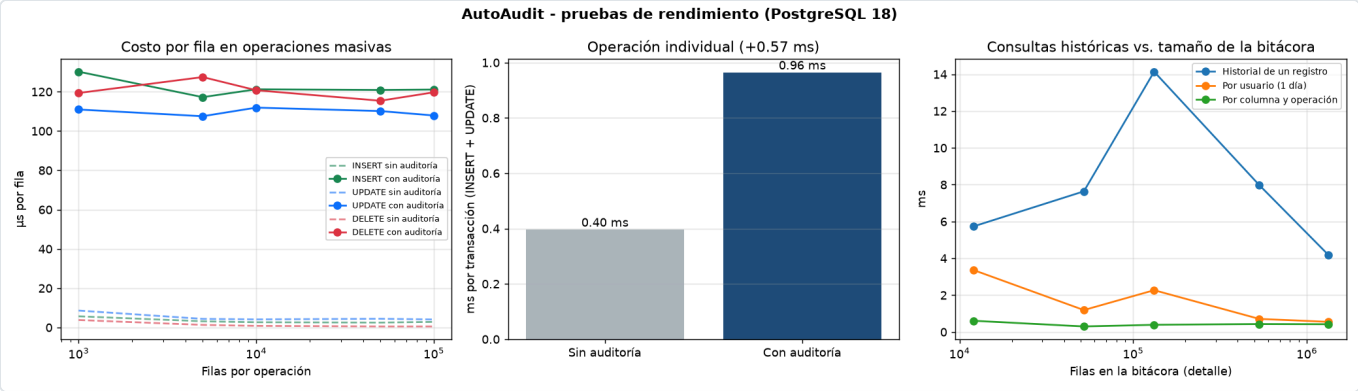


Figura 13. Costo por fila, costo por transacción individual y tiempo de consultas frente al tamaño de la bitácora.

Conclusiones

- **Operación normal** (una fila por transacción, el caso típico de una aplicación): la auditoría agrega menos de 1 ms por transacción de INSERT + UPDATE (entre 0.4 y 0.6 ms en las corridas realizadas en WSL2).
- **Cargas masivas:** el costo es de unos 110 µs por fila auditada, constante con el volumen (no se degrada). La mayor parte corresponde a las llaves foráneas que garantizan la integridad referencial de la bitácora. Para cargas masivas puede excluirse el usuario del proceso, con lo que el costo baja a unos 7 µs por fila.
- **Consultas históricas:** se mantienen por debajo de 15 ms aun con más de 1.3 millones de filas en la bitácora, porque se resuelven por índice.
- Durante el desarrollo estas pruebas detectaron que la bitácora particionada y una consulta a `pg_stat_activity` por transacción triplicaban el costo; ambos se corrigieron.

11. Matriz de cumplimiento

Criterio del enunciado	Cómo se cumple / evidencia
Módulo de configuración	
Interfaz para seleccionar tablas basada en el diccionario	`fn_listar_tablas` ; pantalla Configuración (6.2)
Selección granular de columnas	`aud_columna` ; pantalla de tabla (6.3)
Activación/desactivación por operación	`aud_configuracion.activa` por evento; `test_desactivar_evento_lo_quita_del_trigger`
Persistencia en tablas especializadas	Esquema `autoaudit` (secciones 3 y 4)
Módulo de generación de triggers	
Procesamiento automático de la configuración	`fn_configurar_tabla(..., p_generar := true)`
Generación de código SQL	`_fn_construir_ddl` ; vista previa (figura 6)
Triggers AFTER con valores completos	`AFTER … FOR EACH ROW` ; `fila_anterior` / `fila_nueva`
Gestión de dependencias trigger-tabla	`aud_trigger` , event triggers, `v_estado_triggers` (5.4)
Módulo de bitácora	
Tabla centralizada	`aud_bitacora` + `aud_bitacora_detalle`
Estructura normalizada para consultas	Cabecera/detalle en 3FN e índices (10.2)
Valores antiguos y nuevos	`valor_anterior` , `valor_nuevo` ; `test_update_captura_valor_antiguo_y_nuevo`
Usuario, fecha, hora e IP origen	`aud_usuario` , `aud_estacion` , `fecha_hora` ; `test_insert_registra_todos_los_campos`
Módulo de consulta y reporte	
Búsqueda por usuario	`fn_cambios_por_usuario` ; `test_consulta_por_usuario_historial_completo`
Filtrado por columna	`fn_cambios_por_columna` ; `test_consulta_por_columna_todos_los_cambios`
Visualización cronológica	Línea de tiempo e historial de registro (figuras 9 y 10)
Exportación en formatos estándar	CSV, XLSX, PDF, JSON; `test_exportar_desde_la_web`
Modelo entidad-relación	
Tablas, columnas, eventos, usuarios y estaciones	Sección 3.2
Configuración referencia eventos por columna y tabla	`aud_configuracion` (columna × evento)

Bitácora referencia configuración, usuario y estación	Llaves foráneas (figura 1)
Requisitos técnicos	
PostgreSQL 18 · Ubuntu 26.04 LTS	PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26 · Ubuntu 26.04.1 LTS
PL/pgSQL para generar triggers	`sql/04_generador_triggers.sql`
Servidor e interfaces	Python (FastAPI, Typer)
Restricciones de diseño	
Cada trigger maneja una sola tabla origen	Una función y un trigger por tabla; `test_trigger_generado_es_valido`
Validar existencia antes de activar	`fn_validar_config`; `test_tabla_inexistente`, `test_columna_inexistente`
Preservar datos aunque se elimine el registro	`fila_anterior` jsonb; `test_delete_preserva_los_datos`, `test_drop_table_conserva_bitacora`
Seguridad	
Solo administradores configuran	Sección 8; `test_solo_administradores_configuran`
Sin escritura directa en la bitácora	Sección 8; `test_sin_escritura_directa`, `test_bitacora_inmutable_incluso_para_superusuario`
Privacidad de la IP	`modo_ip`; `test_lector_ve_ip_enmascarada`, `test_modo_ip_hash`
Criterios de aceptación	
Triggers válidos para cada operación configurada	`test_operaciones.py` completo
Cada cambio con todos los campos requeridos	Columnas `NOT NULL`; `test_insert_registra_todos_los_campos`
Consulta por usuario: historial completo	`test_consulta_por_usuario_historial_completo`
Consulta por columna: todos los cambios	`test_consulta_por_columna_todos_los_cambios`
Sin impacto significativo en el rendimiento	Sección 10.2
Configuración refleja el estado actual	`v_estado_triggers`; `test_configuracion_refleja_estado_actual`
Integridad referencial con tablas configuradas	Llaves foráneas y borrado lógico (3.2)
Manejo de errores en generación y ejecución	Sección 9; `test_error_de_generacion_…`, `test_modo_tolerante_…`, `test_modo_estricto_…`
Entregables	
PDF: ER, diccionario, manual, instalación	Este documento (secciones 3, 4, 6 y 7)

Repositorio: consultas al diccionario, scripts, generador, consultas parametrizadas, ejemplos y pruebas	`sql/02…07`, `ejemplos/`, `tests/` (anexo)

12. Anexo: estructura del repositorio y API

```
autoaudit/
├── sql/                    Scripts de base de datos (00 a 07, demo e instalar.sql)
├── app/                    Aplicación web (FastAPI): main, servicios, exportar, auth, db
│   ├── templates/          Pantallas (Jinja2)
│   └── static/             Estilos y JavaScript
├── cli.py                  Interfaz de línea de comandos
├── ejemplos/               Ejemplos de uso en SQL (operaciones, consultas, seguridad y DDL)
├── tests/                  Pruebas funcionales (pytest)
├── pruebas_rendimiento.py  Pruebas de volumen creciente
├── docs/                   Este documento y su generador
├── scripts/                install.sh y utilidades
└── requirements.txt
```

Funciones públicas del esquema autoaudit

Función	Parámetros	Descripción
`fn_actividad_diaria`	p_desde timestamp with time zone, p_hasta timestamp with time zone	
`fn_cambios_por_columna`	p_esquema name, p_tabla name, p_columna name, p_desde timestamp with time zone, p_hasta timestamp with time zone, p_operacion text	Todos los cambios registrados en una columna específica
`fn_cambios_por_usuario`	p_usuario text, p_desde timestamp with time zone, p_hasta timestamp with time zone, p_esquema name, p_tabla name	Historial completo de cambios realizados por un usuario (de BD o de aplicación)
`fn_columnas_pk`	p_oid oid	
`fn_configurar_columna`	p_esquema name, p_tabla name, p_columna name, p_insert boolean, p_update boolean, p_delete boolean, p_generar boolean	Activa/desactiva INSERT, UPDATE y DELETE para una columna específica
`fn_configurar_tabla`	p_esquema name, p_tabla name, p_columnas jsonb, p_generar boolean	Configura columnas y eventos auditados de una tabla y (opcional) genera su trigger
`fn_desactivar_tabla`	p_esquema name, p_tabla name	
`fn_diccionario_datos`	p_esquema name	Diccionario de datos detallado de un esquema (tipos, nulos, llaves, comentarios)
`fn_es_esquema_sistema`	p_esquema name	
`fn_estado_en_fecha`	p_esquema name, p_tabla name, p_pk jsonb, p_fecha timestamp with time zone	Reconstruye el estado de un registro en una fecha a partir de la bitácora

`fn_exigir_admin`		
`fn_generar_trigger`	p_esquema name, p_tabla name, p_forzar boolean	Genera o actualiza el trigger de auditoría de una tabla según su configuración
`fn_historial_registro`	p_esquema name, p_tabla name, p_pk jsonb	Cronología de todos los cambios de un registro identificado por su llave primaria
`fn_ip_visible`	p_ip text	
`fn_listar_columnas`	p_esquema name, p_tabla name	Diccionario de datos: columnas de una tabla y eventos auditados en cada una
`fn_listar_tablas`		Diccionario de datos: tablas de usuario disponibles para auditoría
`fn_previsualizar_trigger`	p_esquema name, p_tabla name	Devuelve el SQL que se generaría para la tabla, sin ejecutarlo
`fn_regenerar_todos`	p_forzar boolean	
`fn_registrar_error`	p_contexto text, p_mensaje text, p_detalle text	
`fn_registrar_usuario`	p_usuario_bd name, p_usuario_app text, p_auditar boolean	
`fn_resumen`	p_desde timestamp with time zone, p_hasta timestamp with time zone	
`fn_set_auditar_estacion`	p_estacion_id integer, p_auditar boolean	
`fn_set_auditar_usuario`	p_usuario_id integer, p_auditar boolean	
`fn_set_parametro`	p_clave text, p_valor text	
`fn_validar_config`	p_tabla_id integer	

Documento generado automáticamente por `docs/generar_documentacion.py` el 30/10/2026 12:13.